```
#Date : 24/07/2026
#DT Batch 3
#Sahil Tagade
#44
#12410016
```

```
#Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine
#dataset, and evaluate its performance using accuracy and a confusion matrix.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report)
import warnings
warnings.filterwarnings("ignore")
```

```
iris = load_iris()
X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df['Flower'] = y
print("Dataset Loaded Successfully")
df.head()
```

```
Dataset Loaded Successfully
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Flower
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
print("Shape of Dataset:")
print(df.shape)
print("\nColumn Names:")
print(df.columns)
print("\nDataset Information:")
df.info()
```

```
Shape of Dataset:
(150, 5)

Column Names:
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'Flower'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   Flower             150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```
print("Missing Values\n")
print(df.isnull().sum())

print("\nDuplicate Rows:", df.duplicated().sum())
```

```
Missing Values

sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
Flower               0
dtype: int64

Duplicate Rows: 1
```

```
df.describe()
```

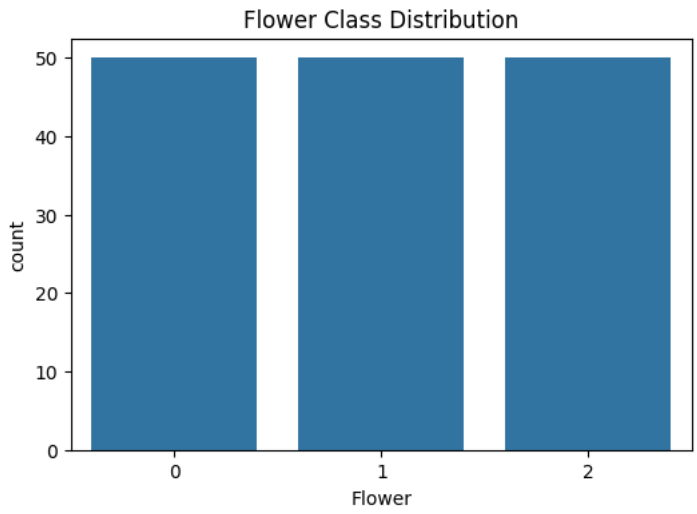
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	Flower
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```
plt.figure(figsize=(6,4))

sns.countplot(
    x="Flower",
    data=df
)

plt.title("Flower Class Distribution")

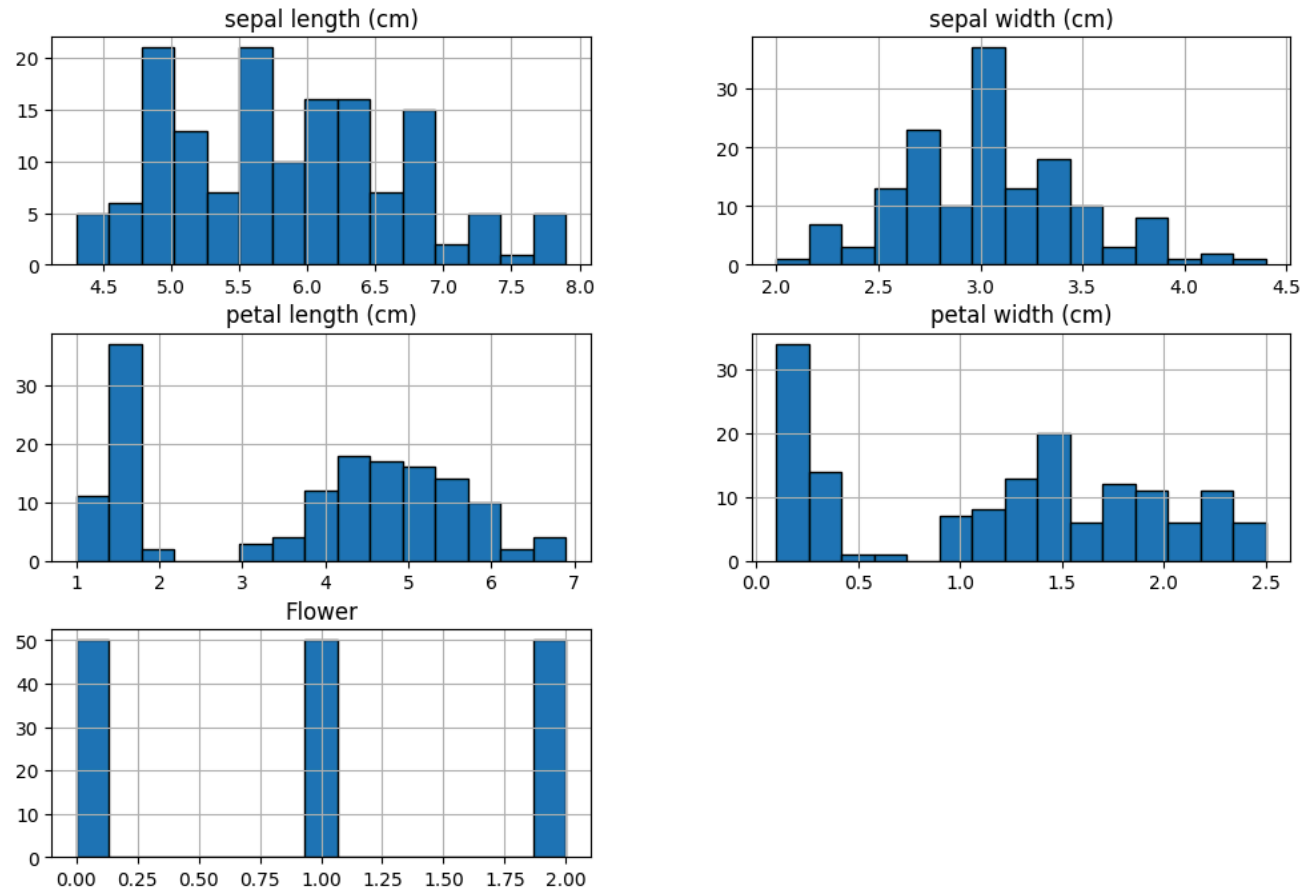
plt.show()
```




```
df.hist(figsize=(12, 8), bins=15, edgecolor='black')

plt.suptitle("Distribution of Iris Features", fontsize=16)

plt.show()
```

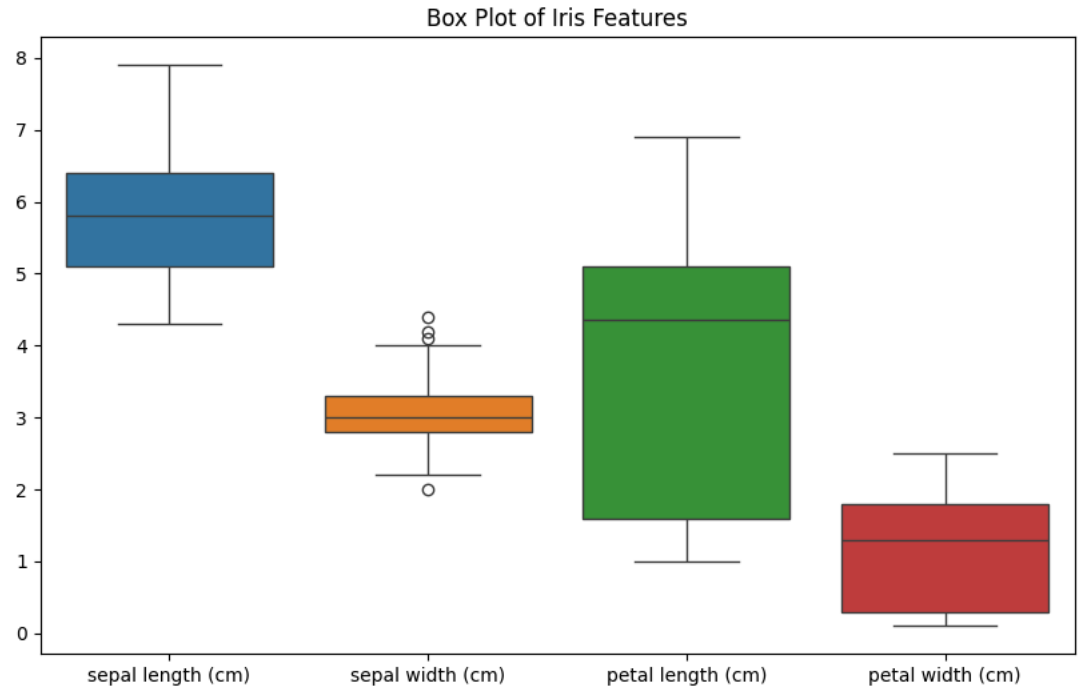



```
plt.figure(figsize=(10,6))

sns.boxplot(data=df.iloc[:, :-1])

plt.title("Box Plot of Iris Features")

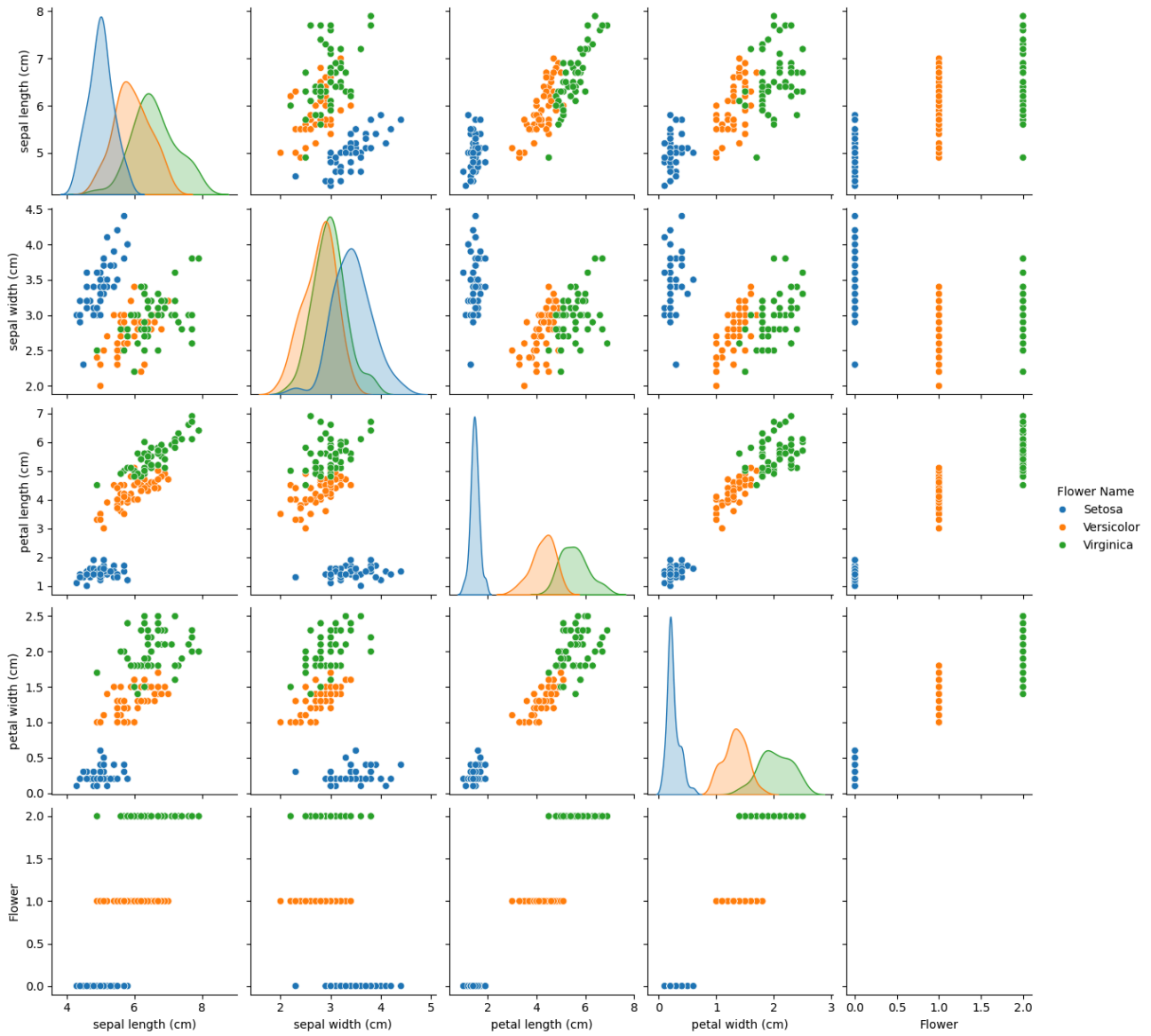
plt.show()
```



```
df["Flower Name"] = df["Flower"].map({
    0: "Setosa",
    1: "Versicolor",
    2: "Virginica"
})

sns.pairplot(
    df,
    hue="Flower Name",
    height=2.5
)

plt.show()
```



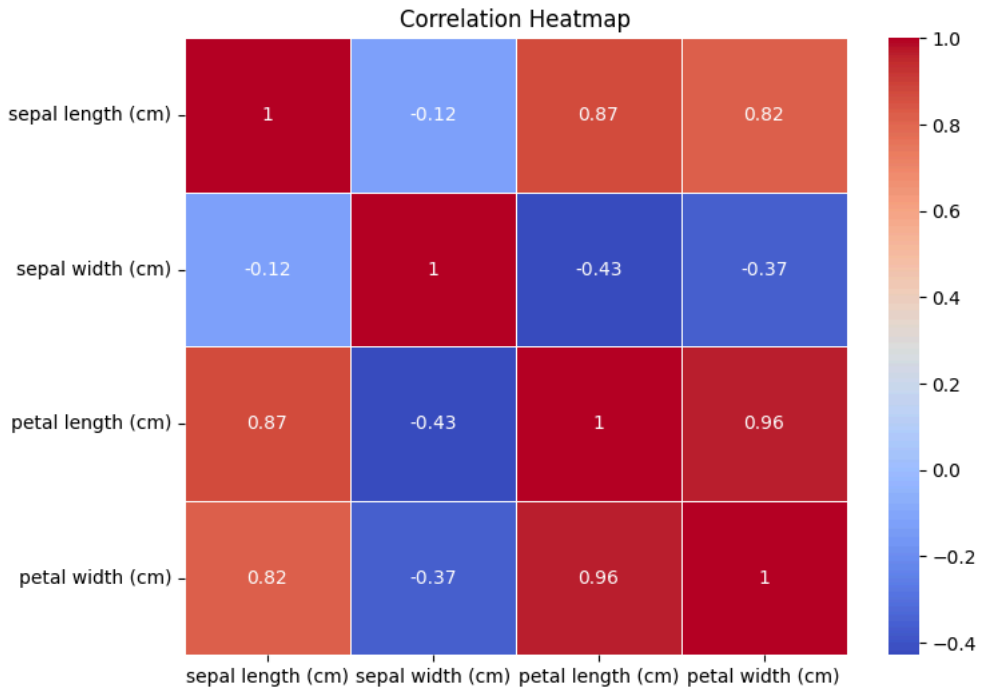

```
plt.figure(figsize=(8,6))

corr = df.iloc[:, :-2].corr()

sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)
```

```
plt.title("Correlation Heatmap")

plt.show()
```

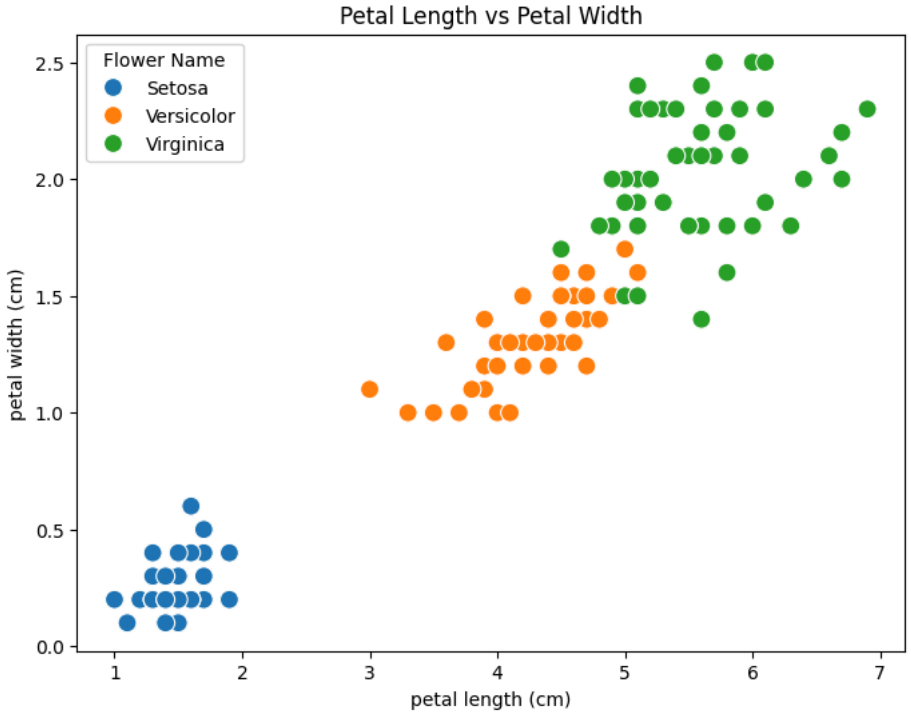



```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df,
    x="petal length (cm)",
    y="petal width (cm)",
    hue="Flower Name",
    s=100
)

plt.title("Petal Length vs Petal Width")

plt.show()
```



```
X = df.iloc[:, :-2]
y = df["Flower"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Samples :", X_train.shape[0])
print("Testing Samples  :", X_test.shape[0])

print("\nTraining Feature Shape :", X_train.shape)
print("Testing Feature Shape  :", X_test.shape)
```

```
Training Samples : 120
Testing Samples  : 30

Training Feature Shape : (120, 4)
Testing Feature Shape  : (30, 4)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("Feature Scaling Completed Successfully!")
```

```
Feature Scaling Completed Successfully!
```

```
mlp = MLPClassifier(
    hidden_layer_sizes=(32, 16),   # Two hidden layers
    activation='relu',             # Activation function
    solver='adam',                 # Optimizer
    learning_rate='adaptive',      # Adaptive learning rate
    max_iter=1000,                 # Maximum training iterations
    random_state=42
)

print("MLP Model Created Successfully!")
```

```
MLP Model Created Successfully!
```

```
mlp.fit(X_train_scaled, y_train)

print("Training Completed Successfully!")
```

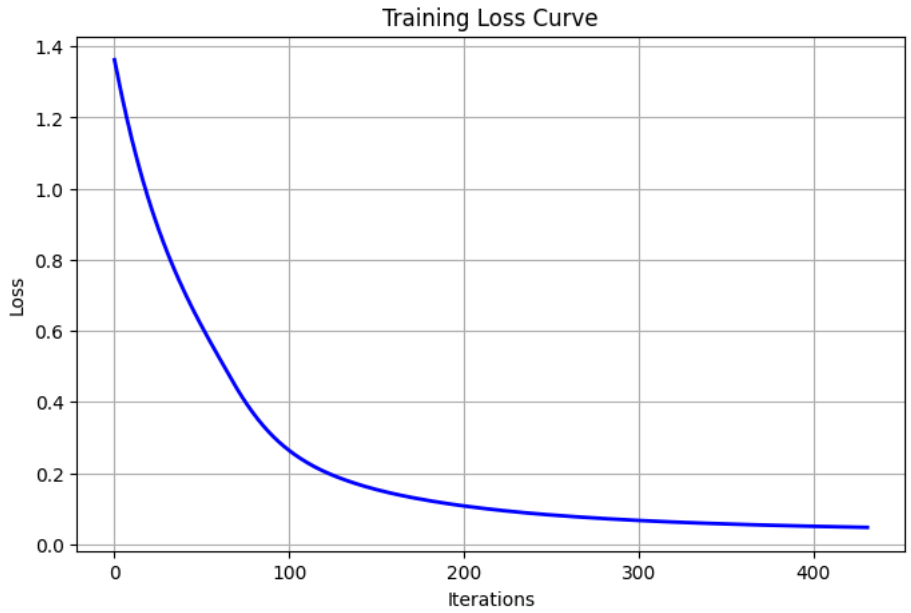
```
Training Completed Successfully!
```

```
plt.figure(figsize=(8,5))

plt.plot(
    mlp.loss_curve_,
    linewidth=2,
    color='blue'
)

plt.title("Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)

plt.show()
```




```
y_pred = mlp.predict(X_test_scaled)

print("Predictions Completed!\n")

print("Actual Labels:\n")
print(y_test.values)

print("\nPredicted Labels:\n")
print(y_pred)
```

```
Predictions Completed!

Actual Labels:

[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted Labels:

[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy of MLP Model")

print(f"\nAccuracy = {accuracy*100:.2f}%")
```

```
Accuracy of MLP Model

Accuracy = 96.67%
```

```
cm = confusion_matrix(
    y_test,
    y_pred
)

print(cm)
```

```
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]
```

```
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    cmap="Blues",
    fmt="d",
    xticklabels=iris.target_names,
```

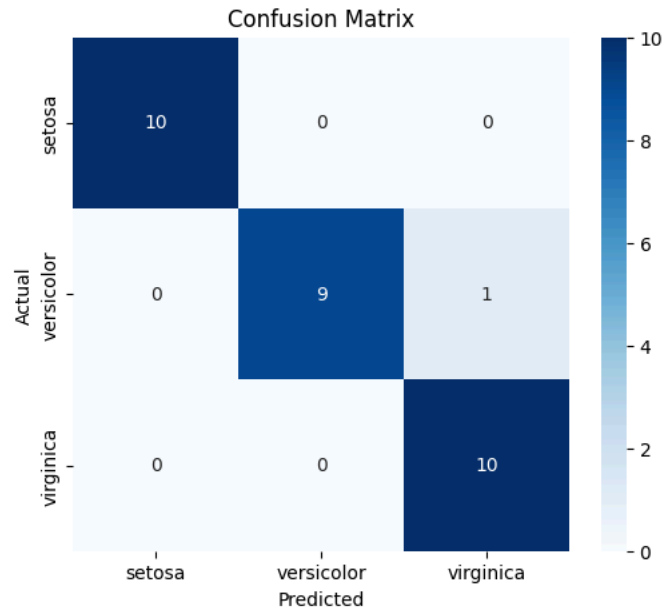
```
    yticklabels=iris.target_names
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```




```
report = classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
)

print(report)
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

```
sample = [[5.1, 3.5, 1.4, 0.2]]

sample_scaled = scaler.transform(sample)

prediction = mlp.predict(sample_scaled)

flower_name = iris.target_names[prediction[0]]

print("Predicted Flower:")

print(flower_name)
```